Lab Assignment-12.5

Name: Goutham Reddy

Hall Ticket no: 2303A51345

Batch No: 20

Task -1:

Prompt:

Task Description #1 (Sorting – Merge Sort Implementation)

• Task: Use AI to generate a Python program that implements the Merge Sort algorithm.

• Instructions:

o Prompt AI to create a function merge_sort(arr) that sorts a list in ascending order.

o Ask AI to include time complexity and space complexity in the function docstring.

o Verify the generated code with test cases.

• Expected Output:

o A functional Python script implementing Merge Sort with proper documentation.

12.5-Assignment.py > ...

```python
#Task-1

def merge_sort(arr):
    """
    Sorts a list using the Merge Sort algorithm.

    Algorithm: Divide-and-conquer approach that recursively splits the list
    into halves until single elements remain, then merges them back in sorted order.

    Time Complexity:
    - Best case: O(n log n)
    - Average case: O(n log n)
    - Worst case: O(n log n)

    Space Complexity: O(n) - requires additional space for merging

    Args:
        arr: List of comparable elements to sort

    Returns:
        Sorted list in ascending order
    """
    # Base case: lists with 0 or 1 element are already sorted
    if len(arr) <= 1:
        return arr

    # Divide: find the middle point
    mid = len(arr) // 2

    # Recursively split left and right halves
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])

    # Merge: combine sorted halves
    return merge(left, right)


def merge(left, right):
    """
    Merges two sorted lists into a single sorted list
```

OUTPUT:

```
PS C:\Users\temba\OneDrive\Desktop\Ai_assig_coding> & C:\Users\temba\AppData\Local\P
g/12.5-Assignment.py
Test 1 - Random list: [64, 34, 25, 12, 22, 11, 90]
Sorted: [11, 12, 22, 25, 34, 64, 90]

Test 2 - Already sorted: [1, 2, 3, 4, 5]
Sorted: [1, 2, 3, 4, 5]

Test 3 - Reverse sorted: [5, 4, 3, 2, 1]
Sorted: [1, 2, 3, 4, 5]

Test 4 - With duplicates: [3, 1, 4, 1, 5, 9, 2, 6]
Sorted: [1, 1, 2, 3, 4, 5, 6, 9]

Test 5 - Single element: [42]
Sorted: [42]
Sorted: [1, 2, 3, 4, 5]

Test 3 - Reverse sorted: [5, 4, 3, 2, 1]
Sorted: [1, 2, 3, 4, 5]

Test 4 - With duplicates: [3, 1, 4, 1, 5, 9, 2, 6]
Sorted: [1, 1, 2, 3, 4, 5, 6, 9]

Test 5 - Single element: [42]
Sorted: [42]
Test 4 - With duplicates: [3, 1, 4, 1, 5, 9, 2, 6]
Sorted: [1, 1, 2, 3, 4, 5, 6, 9]

Test 5 - Single element: [42]
Sorted: [42]
Test 5 - Single element: [42]
Sorted: [42]
Sorted: [42]
Test 6 - Empty list: []
Sorted: []
PS C:\Users\temba\OneDrive\Desktop\Ai_assig_coding> ^C
PS C:\Users\temba\OneDrive\Desktop\Ai_assig_coding> & C:\Users\temba\AppData\Local\P
g/12.5-Assignment.py
Test 1 - Random list: [64, 34, 25, 12, 22, 11, 90]
Sorted: [11, 12, 22, 25, 34, 64, 90]

Test 2 - Already sorted: [1, 2, 3, 4, 5]
Sorted: [1, 2, 3, 4, 5]
```

```
PS C:\Users\temba\OneDrive\Desktop\Ai_assig_coding> & C:\Us
g/12.5-Assignment.py
Test 1 - Random list: [64, 34, 25, 12, 22, 11, 90]
Sorted: [11, 12, 22, 25, 34, 64, 90]

Test 2 - Already sorted: [1, 2, 3, 4, 5]
Sorted: [1, 2, 3, 4, 5]

Test 3 - Reverse sorted: [5, 4, 3, 2, 1]
Sorted: [1, 2, 3, 4, 5]

Test 4 - With duplicates: [3, 1, 4, 1, 5, 9, 2, 6]
Sorted: [1, 1, 2, 3, 4, 5, 6, 9]

Test 5 - Single element: [42]
Sorted: [42]
Test 6 - Empty list: []
Sorted: []
PS C:\Users\temba\OneDrive\Desktop\Ai_assig_coding>
```

Justification:

The array implementation is simple and allows quick access using indexing, but deletion requires shifting elements. The linked list supports dynamic memory allocation and efficient insertions or deletions without shifting. However, searching requires traversal. This comparison highlights performance differences in insertion and deletion efficiency.

Task 2:

Prompt:

Task Description #2 (Searching – Binary Search with AI

Optimization)

• Task: Use AI to create a binary search function that finds a target element in a sorted list.

• Instructions:

o Prompt AI to create a function binary_search(arr, target) returning the index of the target or -1 if not found.

o Include docstrings explaining best, average, and worst-case complexities.

o Test with various inputs.

12.5-Assignment.py > ...

```python
#Task-2

def binary_search(arr, target):
    """
    Searches for a target value in a sorted list using Binary Search algorithm.

    Algorithm: Divide-and-conquer approach that repeatedly divides the search space
    in half by comparing the target with the middle element.

    Time Complexity:
    - Best case: O(1) - target is at the middle position
    - Average case: O(log n) - target found after several divisions
    - Worst case: O(log n) - target at end or not present

    Space Complexity: O(1) - uses only a constant amount of extra space

    Args:
        arr: Sorted list of comparable elements
        target: Element to search for

    Returns:
        Index of target if found, -1 otherwise
    """
    left = 0
    right = len(arr) - 1

    # Continue searching while the search space is valid
    while left <= right:
        # Calculate middle index to avoid overflow
        mid = left + (right - left) // 2

        # Check if target is at middle position
        if arr[mid] == target:
            return mid

        # If target is smaller, search left half
        elif arr[mid] > target:
            right = mid - 1

        # If target is larger, search right half
        else:
            left = mid + 1
```

Output:

```
PS C:\Users\temba\OneDrive\Desktop\Ai_assig_coding> & C:\Users\temba\AppDa
g/12.5-Assignment.py
=== Binary Search Test Cases ===

Test 1 - Find middle element: [1, 3, 5, 7, 9, 11, 13], Target: 7
Result: Index 3

Test 2 - Find first element: [2, 4, 6, 8, 10, 12], Target: 2
Result: Index 0

Test 3 - Find last element: [1, 3, 5, 7, 9, 11], Target: 11
Result: Index 5

Test 4 - Target not found: [10, 20, 30, 40, 50, 60], Target: 35
Result: -1

Test 5 - Single element found: [42], Target: 42
Result: Index 0

Test 6 - Empty list: [], Target: 5
Result: -1

Test 7 - Large list: [0, 2, 4, ..., 98], Target: 50
Result: Index 25
PS C:\Users\temba\OneDrive\Desktop\Ai_assig_coding>
```

Justification:

The Queue ensures requests are processed in FIFO order, maintaining fairness. The Priority Queue improves the system by prioritizing faculty requests over student requests. This demonstrates how selecting appropriate data structures affects system efficiency and real-world applicability in managing institutional workflows.

Task-3:

Prompt:

Task Description #3: Smart Healthcare Appointment Scheduling System

A healthcare platform maintains appointment records containing appointment ID, patient name, doctor name, appointment time, and consultation fee. The system needs to:

1. Search appointments using appointment ID.
2. Sort appointments based on time or consultation fee.

Student Task

• Use AI to recommend suitable searching and sorting algorithms.

• Justify the selected algorithms.

• Implement the algorithms in Python.

```python
12.5-Assignment.py > ...
#Task-3

def binary_search_appointment(appointments, appointment_id):
    """
    Searches for an appointment by appointment_id using Binary Search.

    Assumes appointments list is sorted by appointment_id.

    Time Complexity: O(log n)
    Space Complexity: O(1)

    Args:
        appointments: List of appointment dictionaries sorted by appointment_id
        appointment_id: ID to search for

    Returns:
        Appointment dictionary if found, None otherwise
    """
    left = 0
    right = len(appointments) - 1

    while left <= right:
        mid = left + (right - left) // 2
        mid_id = appointments[mid]['appointment_id']

        if mid_id == appointment_id:
            return appointments[mid]
        elif mid_id > appointment_id:
            right = mid - 1
        else:
            left = mid + 1

    return None


def merge_sort_appointments(appointments, key='appointment_time'):
    """
    Sorts appointments using Merge Sort by specified key.

    Time Complexity: O(n log n)
    Space Complexity: O(n)

    Args:
```

Output:

```
PS C:\Users\temba\OneDrive\Desktop\Ai_assig_coding> & C:\Users\temba\
g/12.5-Assignment.py
=== Smart Healthcare Appointment Scheduling System ===

Test 1 - Search Appointment by ID:
Found: {'appointment_id': 103, 'patient_name': 'Charlie Lee', 'doctor_

Test 2 - Appointments sorted by Time:
  101: Alice Brown - 2024-02-13 11:00
  102: Jane Smith - 2024-02-14 14:30
  105: John Doe - 2024-02-15 10:00
  103: Charlie Lee - 2024-02-15 15:45
  108: Bob Wilson - 2024-02-16 09:15

Test 3 - Appointments sorted by Consultation Fee:
  101: Alice Brown - $100
  102: Jane Smith - $120
  105: John Doe - $150
  103: Charlie Lee - $160
  108: Bob Wilson - $180
PS C:\Users\temba\OneDrive\Desktop\Ai_assig_coding>
```

Justification:

The Stack follows the LIFO principle, making it suitable for ticket escalation systems. The most recent ticket is resolved first, ensuring urgent issues receive attention. Stack operations like push, pop, and peek clearly demonstrate LIFO behavior and structured issue management.

Task 4:

Prompt:

Task Description #4: Railway Ticket Reservation System

Scenario

A railway reservation system stores booking details such as ticket ID, passenger name, train number, seat number, and travel date. The system must:

1. Search tickets using ticket ID.
2. Sort bookings based on travel date or seat number.

Student Task

- Identify efficient algorithms using AI assistance.
- Justify the algorithm choices.
- Implement searching and sorting in Python.

12.5-Assignment.py > ...

```python
#Task-4

def binary_search_ticket(tickets, ticket_id):
    """
    Searches for a ticket by ticket_id using Binary Search.

    Assumes tickets list is sorted by ticket_id.

    Time Complexity: O(log n)
    Space Complexity: O(1)

    Justification: Binary search is optimal for searching in sorted lists.
    With potentially thousands of tickets, O(log n) is far superior to
    linear search O(n).

    Args:
        tickets: List of ticket dictionaries sorted by ticket_id
        ticket_id: ID to search for

    Returns:
        Ticket dictionary if found, None otherwise
    """
    left = 0
    right = len(tickets) - 1

    while left <= right:
        mid = left + (right - left) // 2
        mid_id = tickets[mid]['ticket_id']

        if mid_id == ticket_id:
            return tickets[mid]
        elif mid_id > ticket_id:
            right = mid - 1
        else:
            left = mid + 1

    return None


def merge_sort_tickets(tickets, key='travel_date'):
    """
    Sorts tickets using Merge Sort by specified key.
```

Output:

```
PS C:\Users\temba\OneDrive\Desktop\Ai_assig_coding> & C:\Users\temba\A
g/12.5-Assignment.py
=== Railway Ticket Reservation System ===

Test 1 - Search Ticket by ID:
Found: {'ticket_id': 1003, 'passenger_name': 'Charlie Lee', 'train_num

Test 2 - Tickets sorted by Travel Date:
  1001: Alice Brown - 2024-02-17 (T103)
  1002: Jane Smith - 2024-02-18 (T102)
  1003: Charlie Lee - 2024-02-19 (T102)
  1005: John Doe - 2024-02-20 (T101)
  1008: Bob Wilson - 2024-02-22 (T101)

Test 3 - Tickets sorted by Seat Number:
  1001: Alice Brown - Seat A01
  1005: John Doe - Seat A12
  1002: Jane Smith - Seat B05
  1003: Charlie Lee - Seat B10
  1008: Bob Wilson - Seat C08
PS C:\Users\temba\OneDrive\Desktop\Ai_assig_coding>
```

Justification:

The Hash Table implementation uses chaining to handle collisions efficiently. Insert, search, and delete operations are optimized for fast access. Chaining prevents data loss during collisions and improves performance. This demonstrates practical understanding of hashing techniques and collision resolution strategies.

Task 5:

Prompt:

Task Description #5: Smart Hostel Room Allocation System
A hostel management system stores student room allocation details including student ID, room number, floor, and allocation date. The system needs to:
1. Search allocation details using student ID.
2. Sort records based on room number or allocation date.
Student Task
• Use AI to suggest optimized algorithms.
• Justify the selections.
• Implement the solution in Python

```python
#Task-5

def binary_search_allocation(allocations, student_id):
    """
    Searches for a room allocation by student_id using Binary Search.

    Assumes allocations list is sorted by student_id.

    Time Complexity: O(log n)
    Space Complexity: O(1)

    Justification: Binary search is optimal for searching in sorted lists.
    With potentially hundreds of students, O(log n) is far superior to
    linear search O(n). Provides instant lookup for student records.

    Args:
        allocations: List of allocation dictionaries sorted by student_id
        student_id: ID to search for

    Returns:
        Allocation dictionary if found, None otherwise
    """
    left = 0
    right = len(allocations) - 1

    while left <= right:
        mid = left + (right - left) // 2
        mid_id = allocations[mid]['student_id']

        if mid_id == student_id:
            return allocations[mid]
        elif mid_id > student_id:
            right = mid - 1
        else:
            left = mid + 1

    return None


def merge_sort_allocations(allocations, key='student_id'):
    """
    Sorts allocations using Merge Sort by specified key.
```

Output:

```
PS C:\Users\temba\OneDrive\Desktop\Ai_assig_coding> & C:\Users\temba\AppD
g/12.5-Assignment.py
=== Smart Hostel Room Allocation System ===

Test 1 - Search Allocation by Student ID:
Found: {'student_id': 2018, 'student_name': 'Charlie Lee', 'room_number':

Test 2 - Allocations sorted by Room Number:
  2003: Alice Brown - Room 102 (Floor 1)
  2008: Jane Smith - Room 201 (Floor 2)
  2015: John Doe - Room 305 (Floor 3)
  2018: Charlie Lee - Room 315 (Floor 3)
  2022: Bob Wilson - Room 410 (Floor 4)

Test 3 - Allocations sorted by Allocation Date:
  2003: Alice Brown - 2024-01-02
  2008: Jane Smith - 2024-01-05
  2015: John Doe - 2024-01-10
  2018: Charlie Lee - 2024-01-12
  2022: Bob Wilson - 2024-01-15

Test 4 - Search Non-existent Student:
Result: None
PS C:\Users\temba\OneDrive\Desktop\Ai_assig_coding>
```

Justification:

Different features require different data structures for efficiency. For example, attendance tracking uses arrays, event registration uses lists, library borrowing uses queues, bus scheduling uses linked lists, and cafeteria orders use queues. Choosing appropriate structures improves system organization, performance, and scalability.

Task 6:

Prompt:

Task Description #6: Online Movie Streaming Platform

A streaming service maintains movie records with movie ID, title, genre, rating, and release year. The platform needs to:

1. Search movies by movie ID.
2. Sort movies based on rating or release year.

Student Task

- Recommend searching and sorting algorithms using AI.
- Justify the chosen algorithms.
- Implement Python functions

```python
#Task-6

def binary_search_movie(movies, movie_id):
    """
    Searches for a movie by movie_id using Binary Search.

    Assumes movies list is sorted by movie_id.

    Time Complexity: O(log n)
    Space Complexity: O(1)

    Justification: Binary search is optimal for searching in sorted lists.
    With potentially thousands of movies, O(log n) is far superior to
    linear search O(n). Provides instant lookup for movie records.

    Args:
        movies: List of movie dictionaries sorted by movie_id
        movie_id: ID to search for

    Returns:
        Movie dictionary if found, None otherwise
    """
    left = 0
    right = len(movies) - 1

    while left <= right:
        mid = left + (right - left) // 2
        mid_id = movies[mid]['movie_id']

        if mid_id == movie_id:
            return movies[mid]
        elif mid_id > movie_id:
            right = mid - 1
        else:
            left = mid + 1

    return None


def merge_sort_movies(movies, key='movie_id'):
    """
    Sorts movies using Merge Sort by specified key.

```

Output:

```
PS C:\Users\temba\OneDrive\Desktop\Ai_assig_coding> & C:\Users\temba\AppData\Local\Programs\Python\Python314
g/12.5-Assignment.py
=== Movie Management System ===

Test 1 - Search Movie by ID:
Found: {'movie_id': 2018, 'title': 'Interstellar', 'genre': 'Sci-Fi', 'rating': 8.6, 'release_year': 2014}

Test 2 - Movies sorted by Rating (Highest to Lowest):
  2008: The Dark Knight (Action) - Rating: 9.0
  2015: Inception (Sci-Fi) - Rating: 8.8
  2003: The Matrix (Sci-Fi) - Rating: 8.7
  2018: Interstellar (Sci-Fi) - Rating: 8.6
  2022: Dune (Sci-Fi) - Rating: 8.0

Test 3 - Movies sorted by Release Year:
  2003: The Matrix - 1999
  2008: The Dark Knight - 2008
  2015: Inception - 2010
  2018: Interstellar - 2014
  2022: Dune - 2021

Test 4 - Search Non-existent Movie:
Result: None
PS C:\Users\temba\OneDrive\Desktop\Ai_assig_coding>
```

Justification:

Different features require different data structures for efficiency. For example, attendance tracking uses arrays, event registration uses lists, library borrowing uses queues, bus scheduling uses linked lists, and cafeteria orders use queues. Choosing appropriate structures improves system organization, performance, and scalability.

Task 7:

Prompt:

Task Description #7: Smart Agriculture Crop Monitoring System
An agriculture monitoring system stores crop data with crop ID, crop name, soil moisture level, temperature, and yield estimate. Farmers need to:
1. Search crop details using crop ID.
2. Sort crops based on moisture level or yield estimate.
Student Task
• Use AI-assisted reasoning to select algorithms.
• Justify algorithm suitability.
• Implement searching and sorting in Python.

```python
12.5-Assignment.py > ...
#Task-7

def binary_search_crop(crops, crop_id):
    """
    Searches for a crop by crop_id using Binary Search.

    Assumes crops list is sorted by crop_id.

    Time Complexity: O(log n)
    Space Complexity: O(1)

    Justification: Binary search is optimal for searching in sorted lists.
    With potentially hundreds of crops, O(log n) is far superior to
    linear search O(n). Provides instant lookup for crop records.

    Args:
        crops: List of crop dictionaries sorted by crop_id
        crop_id: ID to search for

    Returns:
        Crop dictionary if found, None otherwise
    """
    left = 0
    right = len(crops) - 1

    while left <= right:
        mid = left + (right - left) // 2
        mid_id = crops[mid]['crop_id']

        if mid_id == crop_id:
            return crops[mid]
        elif mid_id > crop_id:
            right = mid - 1
        else:
            left = mid + 1

    return None


def merge_sort_crops(crops, key='crop_id'):
    """
    Sorts crops using Merge Sort by specified key.
```

Output:

```
PS C:\Users\temba\OneDrive\Desktop\Ai_assig_coding> & C:\Users\temba\AppData\Local\Programs\Python\Python314\python.exe c:/
g/12.5-Assignment.py
=== Smart Agriculture Crop Monitoring System ===

Test 1 - Search Crop by ID:
Found: {'crop_id': 5018, 'crop_name': 'Soybean', 'soil_moisture': 38.7, 'temperature': 26.5, 'yield_estimate': 890}

Test 2 - Crops sorted by Soil Moisture:
  5003: Barley - Soil Moisture: 32.1% (Temp: 20.0°C)
  5015: Wheat - Soil Moisture: 35.5% (Temp: 22.0°C)
  5018: Soybean - Soil Moisture: 38.7% (Temp: 26.5°C)
  5022: Corn - Soil Moisture: 40.0% (Temp: 25.0°C)
  5008: Rice - Soil Moisture: 45.2% (Temp: 28.5°C)

Test 3 - Crops sorted by Yield Estimate:
  5003: Barley - Yield: 650 kg/ha
  5022: Corn - Yield: 780 kg/ha
  5015: Wheat - Yield: 850 kg/ha
  5018: Soybean - Yield: 890 kg/ha
  5008: Rice - Yield: 920 kg/ha

Test 4 - Search Non-existent Crop:
Result: None
PS C:\Users\temba\OneDrive\Desktop\Ai_assig_coding>
```

Justification:

Different features require different data structures for efficiency. For example, attendance tracking uses arrays, event registration uses lists, library borrowing uses queues, bus scheduling uses linked lists, and cafeteria orders use queues. Choosing appropriate structures improves system organization, performance, and scalability.

Task 8:

Prompt:

Task Description #8: Airport Flight Management System

An airport system stores flight information including flight ID, airline name, departure time, arrival time, and status. The system must:

1. Search flight details using flight ID.
2. Sort flights based on departure time or arrival time.

Student Task

• Use AI to recommend algorithms.
• Justify the algorithm selection.
• Implement searching and sorting logic in Python.

12.5-Assignment.py > ...

```python

#Task-8

def binary_search_flight(flights, flight_id):
    """
    Searches for a flight by flight_id using Binary Search.

    Assumes flights list is sorted by flight_id.

    Time Complexity: O(log n)
    Space Complexity: O(1)

    Justification: Binary search is optimal for searching in sorted lists.
    With potentially thousands of flights, O(log n) is far superior to
    linear search O(n). Provides instant lookup for flight records.

    Args:
        flights: List of flight dictionaries sorted by flight_id
        flight_id: ID to search for

    Returns:
        Flight dictionary if found, None otherwise
    """
    left = 0
    right = len(flights) - 1

    while left <= right:
        mid = left + (right - left) // 2
        mid_id = flights[mid]['flight_id']

        if mid_id == flight_id:
            return flights[mid]
        elif mid_id > flight_id:
            right = mid - 1
        else:
            left = mid + 1

    return None


def merge_sort_flights(flights, key='flight_id'):
    """
    Sorts flights using Merge Sort by specified key.
```

Output:

```
PS C:\Users\temba\OneDrive\Desktop\Ai_assig_coding> & C:\Users\tem
g/12.5-Assignment.py
=== Airport Flight Management System ===

Test 1 - Search Flight by ID:
Found: {'flight_id': 4018, 'airline_name': 'FastFly', 'departure_t

Test 2 - Flights sorted by Departure Time:
  4003: SkyAir - Departs 2024-02-20 06:45 (On Time)
  4008: AeroExpress - Departs 2024-02-20 08:15 (On Time)
  4015: SkyAir - Departs 2024-02-20 10:30 (On Time)
  4018: FastFly - Departs 2024-02-20 12:00 (On Time)
  4022: CloudJet - Departs 2024-02-20 14:00 (Delayed)

Test 3 - Flights sorted by Arrival Time:
  4003: SkyAir - Arrives 2024-02-20 10:50 (On Time)
  4008: AeroExpress - Arrives 2024-02-20 12:30 (On Time)
  4015: SkyAir - Arrives 2024-02-20 14:45 (On Time)
  4018: FastFly - Arrives 2024-02-20 16:15 (On Time)
  4022: CloudJet - Arrives 2024-02-20 18:20 (Delayed)

Test 4 - Search Non-existent Flight:
Result: None
PS C:\Users\temba\OneDrive\Desktop\Ai_assig_coding>
```

Justification:

Different features require different data structures for efficiency. For example, attendance tracking uses arrays, event registration uses lists, library borrowing uses queues, bus scheduling uses linked lists, and cafeteria orders use queues. Choosing appropriate structures improves system organization, performance, and scalability.